

C语言两周学习指导

写给刚高考结束的你

恭喜你完成高考，即将踏入计算机专业的大门！C语言作为你的第一门编程语言，是开启编程之旅的钥匙。它不仅是计算机科学的基石，更是理解计算机底层原理的最佳途径。

这份学习计划专为你设计，每天的内容都是精心安排的，包含**重点知识点讲解**、**知识点实战例题（含解题思路）**、**进阶易错点**、**作业练习**（含挑战题）和**自主探索**；每天文末提供**参考答案**，建议先独立完成再对照。开发环境统一推荐使用 **Dev-C++**，降低入门门槛。

学习建议：先看知识点 → 跟做「实战例题」→ 独立完成作业 → 最后翻到当天「参考答案」核对。



整体学习计划

第一周：基础入门与核心语法

天数	主题	主要内容	参考教程
Day 1	环境搭建与 Hello World	Dev-C++安装与操作、第一个C程序、编译链接执行	01-C语言入门 、 02-开发环境搭建 、 03-程序执行原理
Day 2	数据类型与变量	基本数据类型、变量声明与初始化、常量	04-数据类型与变量
Day 3	运算符与表达式	算术运算符、关系运算符、逻辑运算符、优先级	05-运算符与表达式
Day 4	流程控制（上）	if-else语句、条件运算符、简单程序设计	06-流程控制
Day 5	流程控制（下）	switch语句、综合练习、菜单程序	06-流程控制
Day 6	循环结构（上）	while循环、do-while循环、循环应用	07-循环结构
Day 7	循环结构（下）	for循环、循环嵌套、break/continue	07-循环结构

第二周：核心概念与实战应用

天数	主题	主要内容	参考教程
Day 8	数组	数组声明与初始化、数组操作、数组作为函数参数	08-数组与字符串
Day 9	字符串	字符串概念、字符串处理函数、字符串操作	08-数组与字符串
Day 10	函数	函数定义与调用、参数传递、函数原型、递归	09-函数
Day 11	指针（上）	指针概念、取地址与解引用、指针与数组	10-指针
Day 12	指针（下）	指针与函数、动态内存分配、指针高级应用	10-指针 、 11-内存管理
Day 13	结构体	结构体定义与使用、结构体数组、结构体指针	12-结构体与共用体
Day 14	综合实战	三个实战项目、知识回顾与总结	14-综合实战

📖 详细学习内容

Day 1：环境搭建与Hello World

今日目标：安装并熟练使用 Dev-C++，编写、编译、运行并简单调试第一个 C 程序

重点知识点讲解

1.1 为什么推荐 Dev-C++

新生学习 C 语言，最怕一上来就被「编译器 + 编辑器 + 环境变量」劝退。**Dev-C++** 把编辑器、GCC 编译器、调试器打包在一起，安装完成后即可写代码、按快捷键运行，非常适合两周入门。

方案	特点	本计划建议
Dev-C++（推荐）	一站式 IDE，自带 MinGW/GCC	默认使用
VS Code + MinGW	灵活强大，但配置步骤多	入门后再学也可
Visual Studio	功能全、体积大	学校要求再用

本计划主推 **Embarcadero Dev-C++**（Windows）。若你已安装 **Orwell Dev-C++ 5.11**，操作几乎相同，可直接从「1.3 操作步骤」开始。

说明：Dev-C++ 主要面向 Windows。若你使用 macOS/Linux，可临时使用系统自带的 `gcc` / `clang` + 任意编辑器，但课程演示与截图思路仍以 Dev-C++ 为准。

1.2 Dev-C++ 详细安装步骤（Windows）

步骤 1：下载安装包

- 打开浏览器，搜索 `Embarcadero Dev-C++`，或访问 SourceForge / Embarcadero 官方免费页面下载。
- 选择较新的 Windows 安装包（文件名通常类似 `Dev-Cpp x.x.x Setup.exe`）。
- 下载到容易找到的位置（如桌面或「下载」文件夹）。

步骤 2：运行安装程序

1. 双击安装包。若弹出「用户账户控制 (UAC)」提示，点击**是**。
2. 选择安装语言（可先选 English，装好后可在软件内改中文界面；若安装包提供中文也可直接选中文）。
3. 阅读许可协议，勾选同意后继续。

步骤 3：选择安装路径（重要）

1. 建议安装到简单路径，例如：`C:\Dev-Cpp`
2. **尽量避免**：
 - 路径中含中文（如 `C:\编程工具\Dev-Cpp`）
 - 路径中含空格过多的复杂目录（个别旧环境可能出问题）
3. 其余选项保持默认即可（创建桌面快捷方式建议勾选）。
4. 点击安装，等待进度条完成，最后点击完成/Finish。

步骤 4：首次启动检查

1. 从桌面快捷方式或开始菜单打开 **Dev-C++**。
2. 若首次运行弹出编译器检测 / 配置向导：
 - 选择自带的 **MinGW / TDM-GCC**（以安装包实际名称为准）
 - 点击检查或确定，确保能检测到 `gcc.exe`
3. 菜单大致位置（不同版本用词可能略有差异）：
 - `Tools` → `Compiler Options`：编译器相关
 - `Tools` → `Editor Options`：编辑器字体、Tab 宽度（建议 Tab=4）

步骤 5：界面语言与编码建议（可选）

1. 若菜单是英文，可在 `Tools` → `Environment Options` 中查找 Language，改为中文（若有语言包）。
2. 源文件建议统一使用 **UTF-8** 保存，减少中文注释/输出乱码概率。
3. 字体可调大一些（如 Consolas / 微软雅黑，字号 12~14），长时间写代码更轻松。

安装成功的标志：能打开 Dev-C++，且之后能编译运行 Hello World（见下一节）。

1.3 Dev-C++ 操作步骤：从零到 Hello World

方式 A：直接新建源文件（最快，推荐第一天使用）

1. 菜单：`File` → `New` → `Source File`（或按 `Ctrl+N`）。
2. 在空白编辑区输入下面的程序。
3. 菜单：`File` → `Save As...`，建议先建文件夹，例如：`D:\C_Learning\day01\`
4. 文件名设为 `hello.c`（扩展名必须是 `.c`，不要存成 `.cpp` 除非你明确在写 C++）。
5. 编译：按 `F9`，或菜单 `Execute` → `Compile`。
 - 下方编译日志出现 `Compilation successful` / 成功，即表示通过。
6. 运行：按 `F10`，或 `Execute` → `Run`。
7. 更常用：按 `F11`（Compile & Run，先编译再运行）。

方式 B：新建控制台项目（更接近正式工程）

1. `File` → `New` → `Project...`
2. 选择 **Console Application**，语言选 **C**
3. 输入项目名（如 `HelloWorld`），选择保存目录

4. 在自动生成的 `main.c` 中改写代码，同样用 `F9` / `F10` / `F11`

Hello World 程序：

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

代码解析：

- `#include <stdio.h>`：引入标准输入输出库，才能使用 `printf` / `scanf`
- `int main(void)`：程序入口；`void` 表示暂时不接收命令行参数
- `printf(...)`：向屏幕输出；`\n` 表示换行
- `return 0`：返回给操作系统，`0` 通常表示正常结束

运行后你会看到：黑色控制台窗口打印 `Hello, World!`。部分版本窗口一闪而过——可在程序末尾加一句：

```
system("pause"); /* 需要 #include <stdlib.h>, 仅用于本地暂停, 正式作业可不写 */
```

或养成习惯：看完输出后看编译日志区，不要立刻关窗口。

1.4 读懂编译错误（必会）

故意把代码改错，例如删掉分号：

```
printf("Hello, World!\n")
```

按 `F9` 后，下方会提示类似 `error: expected ';' before ...`，并给出**行号**。双击错误信息通常能跳到对应行。

习惯：先看第一条错误 → 改 → 再编译。不要一次改十处却不重新编译。

1.5 简单调试（断点与单步）

1. 在某一左侧空白处单击，出现红点 = **断点**（程序运行到这里会暂停）。
2. 使用调试运行（常见为菜单 `Debug` → `Debug` / 快捷键，视版本而定；有的版本用 `F5` 或工具栏虫子图标）。
3. 常用操作：
 - **Step Into / 逐过程进入**：进入函数内部（常见 `F7`）
 - **Step Over / 逐过程跳过**：执行当前行但不进入函数（常见 `F8`）
 - **Continue / 继续**：跑到下一个断点

4. 在调试面板中查看变量当前值（CPU 窗口 / Debug 窗口，名称因版本而异）。

第一天目标：会设断点、能单步、能看到变量变化即可，不必精通全部调试菜单。

1.6 程序如何变成可执行文件（加深）

写出 `.c` 源文件后，Dev-C++ 背后大致做了三件事：

1. **编译 (Compile)**：把 C 代码翻译成目标文件（`.o` / `.obj`）
2. **链接 (Link)**：把目标文件和标准库拼成可执行文件（`.exe`）

3. **执行 (Run)** : 操作系统加载 `.exe` 运行

因此: 改了代码却觉得「没变化」, 多半是**忘了重新编译**。养成 `F11` 的习惯。

1.7 补充: 命令行方式 (了解即可)

Dev-C++ 自带的 `gcc` 也可在命令行使用 (需把其 `bin` 目录加入 PATH, 或在「Dev-C++ 终端」中操作)。了解下列命令有助于理解 IDE 在做什么:

```
gcc hello.c -o hello.exe
hello.exe
```

本计划日常仍以 Dev-C++ 图形界面为主。

知识点实战例题

例题1: 格式化输出与转义字符

题目: 编写程序, 输出如下三行 (注意对齐与换行):

```
Name:    Alice
ID:      2026001
Score:   95
```

解题思路:

1. 需要多次 `printf`, 或一次 `printf` 中写多个 `\n`
2. 列对齐可用多个空格, 或使用 `\t` 制表符 (制表位取决于控制台设置)
3. 先写第一行跑通, 再复制改成三行, 最后微调空格

完整代码:

```
#include <stdio.h>

int main(void) {
    printf("Name:\tAlice\n");
    printf("ID:\t2026001\n");
    printf("Score:\t95\n");
    return 0;
}
```

要点/易错:

- `\n` 只换行, 不会自动对齐; 对齐靠空格或 `\t`
- 字符串里要输出 `\` 本身需写成 `\\`
- 改完务必重新 `F11` 编译运行

例题2: scanf 读入并计算

题目: 从键盘输入长和宽 (整数), 输出矩形周长与面积。

解题思路:

1. 定义两个 `int` 变量存放长、宽

2. 用 `scanf` 读入, 变量前加 `&`
3. 周长 = `(长 + 宽) * 2`, 面积 = `长 * 宽`
4. 用 `printf` 输出结果

完整代码:

```
#include <stdio.h>

int main(void) {
    int length, width;
    printf("请输入长和宽 (用空格分隔) : ");
    scanf("%d %d", &length, &width);
    printf("周长 = %d\n", (length + width) * 2);
    printf("面积 = %d\n", length * width);
    return 0;
}
```

要点/易错:

- 忘写 `&` 是新手最常见错误之一
- `scanf` 格式个数要与变量个数一致
- 输入时用空格或回车分隔均可

作业练习

先独立完成, 做完再到本节文末「参考答案」对照。

练习1: 完成环境验证

- 按上文安装 Dev-C++
- 新建 `hello.c`, 编译运行成功
- 故意制造一个语法错误, 学会根据行号定位并修复

练习2: 输出个人信息

- 创建 `info.c`
- 用多次 `printf` 输出姓名、拟报专业、兴趣
- 在 Dev-C++ 中 `F11` 运行

练习3: 计算两个数的和

- 创建 `sum.c`
- 使用 `scanf` 输入两个整数, 输出它们的和
- 注意: `scanf("%d", &a);` 中的 `&` 不能丢

挑战题: 个人信息卡片

- 使用 `\n`、`\t` 和若干 `=` / `-` 字符, 打印一个整齐的「名片」边框
- 要求至少 5 行输出, 左右对齐看起来像卡片

常见错误与调试提示

- 错误: 找不到编译器 / `gcc` 相关报错

- **解决**：重装时勾选自带编译器； `Tools` → `Compiler Options` 中确认编译器路径指向安装目录下的 MinGW
- **错误**：中文输出乱码
 - **解决**：源文件改为 UTF-8 保存；或临时少用中文，先用英文验证环境；控制台也可试 `chcp 65001`
- **错误**：程序一闪而过看不清输出
 - **解决**：用 `F11` 后查看；或调试运行；或临时 `system("pause")`
- **错误**：保存成了 `.cpp` 或 `.txt`
 - **解决**：另存为时文件类型选 All files，文件名写 `xxx.c`
- **错误**： `scanf` 输入后结果不对
 - **解决**：检查格式串是否与类型匹配，变量前是否写了 `&`

自主探索

1. 在 Dev-C++ 中尝试修改字体、主题，找到自己舒服的编辑环境
2. 查一查 `\n`、`\t`、`\\`、`\"` 转义字符的含义并写个小 demo
3. 搜索了解 C 语言简史 (K&R、ANSI C、C99)，写 200~300 字笔记

参考答案

练习1 参考答案 (hello.c)

思路：最小可运行程序；故意删分号观察报错行号，再改回。

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

练习2 参考答案 (info.c)

```
#include <stdio.h>

int main(void) {
    printf("姓名：张三\n");
    printf("拟报专业：计算机科学与技术\n");
    printf("兴趣：编程、篮球\n");
    return 0;
}
```

练习3 参考答案 (sum.c)

思路：定义两个变量 → `scanf` 读入（带 `&`）→ 求和输出。

```
#include <stdio.h>

int main(void) {
    int a, b;
    printf("请输入两个整数: ");
    scanf("%d %d", &a, &b);
    printf("%d + %d = %d\n", a, b, a + b);
    return 0;
}
```

挑战题参考答案

思路：先用 `=` 打出上下边框，中间用 `printf` 输出字段，注意每行宽度大致一致。

```
#include <stdio.h>

int main(void) {
    printf("=====\n");
    printf("| Name:\tXiaoming |\n");
    printf("| Major:\tCS      |\n");
    printf("| Hobby:\tCoding  |\n");
    printf("=====\n");
    return 0;
}
```

Day 2：数据类型与变量

今日目标：理解C语言的数据类型系统，学会定义和使用变量

重点知识点讲解

2.1 基本数据类型

类型	关键字	大小（字节）	范围
字符型	char	1	-128~127
短整型	short	2	-32768~32767
整型	int	4	-21亿~21亿
长整型	long	4或8	取决于系统
单精度浮点	float	4	约±3.4×10 ³⁸
双精度浮点	double	8	约±1.7×10 ³⁰⁸

2.2 变量的声明与初始化


```
// 声明并初始化
int age = 25;
float height = 1.75f;
char gender = 'M';

// 先声明后赋值
int score;
score = 95;

// 多个变量同时声明
int a = 1, b = 2, c = 3;
```

2.3 变量命名规则

1. 必须以字母或下划线开头
2. 只能包含字母、数字和下划线
3. 不能是C语言关键字
4. 区分大小写

2.4 常量

```
// 使用#define定义宏常量
#define PI 3.14159
#define MAX_SIZE 100

// 使用const定义常量
const float GRAVITY = 9.8f;
```

2.5 进阶：sizeof、符号与格式匹配（易错）

```
printf("%zu\n", sizeof(int)); /* 打印类型占用字节数 */
unsigned int u = 1;
int s = -1;
/* 有符号与无符号混用时可能出现“惊人”的比较结果，初学阶段尽量统一用 int */
```

要点：

1. `sizeof`：得到类型或变量的字节数，结果类型与平台有关，打印时常用 `%zu`（若编译器不支持可先用 `%d` 并强转 `(int)sizeof(int)` 做实验）。
2. **初始化**：局部变量若不初始化，值是不定的（未定义行为），务必养成初始化习惯。
3. `printf` / `scanf` **格式必须匹配类型**：`int` 用 `%d`，`float` 用 `%f`，`double` 用 `%lf`（`scanf`） / `%f` 或 `%lf`（`printf`，C99 起对 `double` 也常用 `%f`）。类型与格式不匹配是隐蔽 bug 的常见来源。
4. **整型提升（了解）**：`char`、`short` 参与运算时，往往会先提升为 `int` 再计算。

知识点实战例题

例题1：常量与浮点计算

题目：用 `#define` 定义税率 `TAX_RATE` 为 `0.13`，输入商品不含税价格（浮点数），输出税额与含税价（保留两位小数）。

解题思路：

1. 用宏定义税率，避免魔法数字散落代码中
2. `scanf("%f", &price)` 读入价格
3. 税额 = 价格 × 税率；含税价 = 价格 + 税额
4. `%.2f` 控制小数位

完整代码：

```
#include <stdio.h>

#define TAX_RATE 0.13

int main(void) {
    float price, tax, total;
    printf("请输入不含税价格: ");
    scanf("%f", &price);
    tax = price * TAX_RATE;
    total = price + tax;
    printf("税额: %.2f\n", tax);
    printf("含税价: %.2f\n", total);
    return 0;
}
```

要点/易错：

- `float` 用 `%f`；不要写成 `%d`
- 宏定义一般不加分号：`#define TAX_RATE 0.13` 正确

例题2：字符与 ASCII

题目：输入一个大写字母，输出它的小写形式及其 ASCII 码。

解题思路：

1. 大写转小写：`ch + ('a' - 'A')`，或直接 `ch + 32`（了解即可）
2. 用 `%c` 读字符、`%d` 打印 ASCII（把 `char` 当整数看）

完整代码：

```
#include <stdio.h>

int main(void) {
    char ch;
    printf("请输入一个大写字母: ");
    scanf(" %c", &ch); /* 前面空格可吃掉残留空白 */
    char lower = ch + ('a' - 'A');
    printf("小写: %c, ASCII: %d\n", lower, lower);
    return 0;
}
```

要点/易错：

- 字符用单引号 `'A'`，字符串用双引号 `"A"`
- `scanf("%c")` 前加空格更稳妥，避免读到上次留下的换行

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：计算圆的周长和面积

- 输入半径
- 计算并输出周长和面积
- 使用 `#define` 定义 π

练习2：温度转换

- 输入华氏温度
- 转换为摄氏温度并输出（公式： $C = (F - 32) * 5 / 9$ ）

练习3：变量作用域实验

- 定义全局变量和局部变量
- 观察变量的可见范围（可在不同函数中打印同名/全局变量）

挑战题：类型大小与溢出观察

1. 用 `sizeof` 打印 `char/short/int/long/float/double` 的字节数
2. 令 `unsigned char x = 255; x = x + 1;`，观察输出（理解回绕）
3. 思考：为什么银行/金额计算要特别小心浮点类型？

常见错误与调试提示

- **错误：**使用未初始化的变量
 - **解决：**养成初始化变量的好习惯，避免未定义行为
- **错误：**浮点精度问题
 - **解决：**浮点数比较时要小心，使用合理的精度范围
- **错误：**类型溢出
 - **解决：**注意数据类型的范围，选择合适的类型
- **错误：**`scanf` 写了 `%d` 却传入 `float` 变量
 - **解决：**改格式或改变量类型，保持一致

自主探索

1. 使用 `sizeof` 运算符查看不同数据类型的大小
2. 尝试将整数转换为字符，观察ASCII码的变化
3. 搜索了解 `auto`、`static`、`extern`、`register` 存储类型的区别

参考答案

练习1 参考答案

思路：宏定义 π → 读入半径 → 周长 `2*PI*r`，面积 `PI*r*r` → 保留两位小数输出。

```
#include <stdio.h>

#define PI 3.14159

int main(void) {
    float radius;
    printf("请输入圆的半径: ");
    scanf("%f", &radius);
    printf("圆的周长: %.2f\n", 2 * PI * radius);
    printf("圆的面积: %.2f\n", PI * radius * radius);
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    float f, c;
    printf("请输入华氏温度: ");
    scanf("%f", &f);
    c = (f - 32) * 5 / 9;
    printf("摄氏温度: %.2f\n", c);
    return 0;
}
```

练习3 参考答案

思路：全局变量在所有函数可见；局部变量只在定义它的函数/块内可见。

```
#include <stdio.h>

int global_n = 100; /* 全局 */

void show(void) {
    int local_n = 1; /* 局部, 仅 show 内可见 */
    printf("in show: global=%d, local=%d\n", global_n, local_n);
}

int main(void) {
    int local_n = 2;
    printf("in main: global=%d, local=%d\n", global_n, local_n);
    show();
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
```

```

printf("char=%d short=%d int=%d long=%d float=%d double=%d\n",
      (int)sizeof(char), (int)sizeof(short), (int)sizeof(int),
      (int)sizeof(long), (int)sizeof(float), (int)sizeof(double));

unsigned char x = 255;
x = x + 1;
printf("unsigned char 255+1 = %u (回绕到0) \n", (unsigned)x);

/* 思考：浮点有精度误差，金额更适合用整数分/专用小数类型 */
return 0;
}

```

Day 3: 运算符与表达式

今日目标：掌握C语言的各种运算符，理解表达式的计算规则

重点知识点讲解

3.1 算术运算符

```

int a = 10, b = 3;
a + b; // 加法: 13
a - b; // 减法: 7
a * b; // 乘法: 30
a / b; // 除法: 3 (整数除法, 截断小数)
a % b; // 取余: 1

```

3.2 关系运算符

```

a > b; // 大于: 1 (真)
a < b; // 小于: 0 (假)
a >= b; // 大于等于: 1
a <= b; // 小于等于: 0
a == b; // 等于: 0
a != b; // 不等于: 1

```

3.3 逻辑运算符

```

(a > 0) && (b > 0); // 逻辑与: 两个都为真才为真
(a > 0) || (b > 0); // 逻辑或: 一个为真即为真
!(a > 0);           // 逻辑非: 取反

```

3.4 自增自减运算符

```

int x = 5;
int y = ++x; // 前缀自增: 先加1再使用, y=6, x=6
int z = x++; // 后缀自增: 先使用再加1, z=6, x=7

```

3.5 运算符优先级

- 括号 > 单目运算符 > 算术运算符 > 关系运算符 > 逻辑运算符 > 赋值运算符

拿不准时：**加括号**，可读性远比炫技重要。

3.6 进阶：短路求值与赋值陷阱

短路求值：

- `&&`：左边为假则不再计算右边
- `||`：左边为真则不再计算右边

```
int x = 0;
(x != 0) && (10 / x > 1); /* x==0 时不会执行除法，避免除零 */
```

赋值 vs 比较：

```
if (x = 0) { /* 这是赋值！条件几乎总是假，且改写了 x */ }
if (x == 0) { /* 这才是比较 */ }
```

逗号运算符（了解）：`a = (1, 2, 3);` 结果为 `3`，日常少用，读代码时能认出即可。

知识点实战例题

例题1：整数除法与取余

题目：输入总秒数，换算成「时:分:秒」输出（例如 `3661` → `1:1:1`）。

解题思路：

1. 小时 = 总秒 / 3600
2. 剩余秒 = 总秒 % 3600
3. 分钟 = 剩余 / 60，秒 = 剩余 % 60

完整代码：

```
#include <stdio.h>

int main(void) {
    int total, h, m, s;
    printf("请输入总秒数: ");
    scanf("%d", &total);
    h = total / 3600;
    m = (total % 3600) / 60;
    s = total % 60;
    printf("%d:%d:%d\n", h, m, s);
    return 0;
}
```

要点/易错：

- `/` 对整数是截断除法；要小数需写成浮点运算
- `%` 只能用于整数

例题2：前缀/后缀自增

题目：设 `x=5`，分别计算并打印 `a=++x` 与 `b=x++` 后 `a,b,x` 的值（分两段实验，每次从 `x=5` 开始）。

解题思路：

1. `++x`：先加再取值
2. `x++`：先取值再加
3. 分开写两段，避免在同一表达式里多次改 `x`

完整代码：

```
#include <stdio.h>

int main(void) {
    int x = 5, a, b;
    a = ++x;
    printf("a=++x → a=%d, x=%d\n", a, x);
    x = 5;
    b = x++;
    printf("b=x++ → b=%d, x=%d\n", b, x);
    return 0;
}
```

要点/易错：不要在一个表达式里对同一变量多次 `++/--`（未定义行为）。

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：判断闰年

- 输入年份
- 判断是否为闰年（能被4整除但不能被100整除，或能被400整除）

练习2：计算BMI指数

- 输入体重(kg)和身高(m)
- 计算BMI并判断等级（可参考：<18.5偏瘦，18.5~24正常，24~28偏胖，≥28肥胖）

练习3：位运算权限控制

- 用一个 `unsigned int flags` 表示权限
- 约定：第0位=读，第1位=写，第2位=执行
- 练习：打开某权限、关闭某权限、判断是否拥有某权限（`|`、`&`、`~`）

挑战题：

1. 不用临时变量，仅用算术或异或交换两个整数，并说明各自局限
2. 写一段代码验证短路：在 `&&` / `||` 右侧放入带 `printf` 的表达式，观察是否打印

常见错误与调试提示

- **错误：**使用 `=` 代替 `==`
 - **解决：**将常量放在前面，如 `if (10 == x)`，误写为 `=` 会产生编译错误
- **错误：**整数除法结果不符合预期

- **解决**：使用强制类型转换实现浮点除法，如 `(float)a / b`
- **错误**：在同一表达式中多次修改同一变量
 - **解决**：这种行为未定义，应避免

自主探索

1. 搜索了解短路求值的概念，编写代码验证
2. 尝试使用位运算实现交换两个数（不使用临时变量）
3. 搜索了解运算符优先级表，总结常用的优先级规则

参考答案

练习1 参考答案

思路：闰年条件写成一个表达式，结果为 1/0，再用 if 输出。

```
#include <stdio.h>

int main(void) {
    int year;
    printf("请输入年份: ");
    scanf("%d", &year);
    int is_leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    if (is_leap) {
        printf("%d年是闰年\n", year);
    } else {
        printf("%d年不是闰年\n", year);
    }
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    float weight, height, bmi;
    printf("请输入体重(kg)和身高(m): ");
    scanf("%f %f", &weight, &height);
    bmi = weight / (height * height);
    printf("BMI=%.2f, ", bmi);
    if (bmi < 18.5f) {
        printf("偏瘦\n");
    } else if (bmi < 24.0f) {
        printf("正常\n");
    } else if (bmi < 28.0f) {
        printf("肥胖\n");
    } else {
        printf("肥胖\n");
    }
    return 0;
}
```


练习3 参考答案

```
#include <stdio.h>

#define READ  (1u << 0)
#define WRITE (1u << 1)
#define EXEC  (1u << 2)

int main(void) {
    unsigned int flags = 0;
    flags |= READ;           /* 打开读 */
    flags |= WRITE;          /* 打开写 */
    flags &= ~WRITE;         /* 关闭写 */
    if (flags & READ) {
        printf("拥有读权限\n");
    }
    if (!(flags & WRITE)) {
        printf("没有写权限\n");
    }
    flags |= EXEC;
    printf("flags=%u\n", flags);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
    int a = 3, b = 5;
    /* 算术交换: 注意溢出风险 */
    a = a + b;
    b = a - b;
    a = a - b;
    printf("算术交换后 a=%d b=%d\n", a, b);

    a = 3;
    b = 5;
    a ^= b;
    b ^= a;
    a ^= b;
    printf("异或交换后 a=%d b=%d\n", a, b);

    int x = 0;
    (x != 0) && printf("不会打印 (短路) \n");
    (x == 0) || printf("也不会打印 (短路) \n");
    (x == 0) && printf("会打印\n");
    return 0;
}
```

Day 4: 流程控制（上）

今日目标：学会使用if-else语句实现条件判断

重点知识点讲解

4.1 基本if语句

```
if (条件) {  
    // 条件为真时执行  
}
```

4.2 if-else语句

```
if (条件) {  
    // 条件为真时执行  
} else {  
    // 条件为假时执行  
}
```

4.3 嵌套if-else语句

```
if (score >= 90) {  
    printf("优秀");  
} else if (score >= 80) {  
    printf("良好");  
} else if (score >= 60) {  
    printf("及格");  
} else {  
    printf("不及格");  
}
```

4.4 条件运算符（三元运算符）

```
int max = (a > b) ? a : b;
```

4.5 进阶：Dangling else 与输入合法性

Dangling else（悬挂 else）：else 总是匹配最近的、尚未匹配的 if。用花括号消除歧义：

```
if (a)  
    if (b)  
        printf("b");  
else    /* 这个 else 匹配的是内层 if (b), 不是 if (a)! */  
    printf("?");
```

输入合法性：用户可能输入负数、超范围数据。关键程序应对 scanf 返回值或范围做检查：

```
if (scanf("%d", &score) != 1) {  
    printf("输入无效\n");  
    return 1;  
}
```

知识点实战例题

例题1：成绩等级

题目：输入百分制成绩，输出等级：≥90 优秀，≥80 良好，≥60 及格，否则不及格。非法成绩（<0 或 >100）提示错误。

解题思路：

1. 先校验范围
2. 用 `if-else if` 从高到低判断（注意顺序）
3. 每个分支只打印一种结果

完整代码：

```
#include <stdio.h>  
  
int main(void) {  
    int score;  
    printf("请输入成绩(0-100): ");  
    if (scanf("%d", &score) != 1) {  
        printf("输入无效\n");  
        return 1;  
    }  
    if (score < 0 || score > 100) {  
        printf("成绩超出范围\n");  
    } else if (score >= 90) {  
        printf("优秀\n");  
    } else if (score >= 80) {  
        printf("良好\n");  
    } else if (score >= 60) {  
        printf("及格\n");  
    } else {  
        printf("不及格\n");  
    }  
    return 0;  
}
```

要点/易错：若先写 `>=60` 再写 `>=90`，高分会被误判为及格。

例题2：三元运算符求较大值

题目：输入两个整数，用三元运算符输出较大者。

解题思路： `max = (a > b) ? a : b;`

完整代码：

```
#include <stdio.h>

int main(void) {
    int a, b;
    printf("输入两个整数: ");
    scanf("%d %d", &a, &b);
    printf("较大值=%d\n", (a > b) ? a : b);
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：判断奇偶性

- 输入一个整数
- 判断是奇数还是偶数

练习2：判断闰年（使用if-else实现）

练习3：简单计算器（加减乘除）

- 输入两个数和运算符
- 使用if-else判断运算符并计算
- 注意处理除数为 0

挑战题：

1. 实现分段函数： $x < 0 \rightarrow -1$ ； $x == 0 \rightarrow 0$ ； $x > 0 \rightarrow 1$ （也可用三元运算符嵌套试写）
2. 输入三个整数，输出从大到小排序后的结果（只用 if-else，不用数组）

常见错误与调试提示

- **错误：**if后面误加分号
 - **解决：**`if (x > 0); { ... }`会导致块总是执行
- **错误：**条件表达式逻辑错误
 - **解决：**使用括号明确优先级，逐步调试
- **错误：**嵌套过多导致逻辑混乱
 - **解决：**简化条件，使用函数封装复杂逻辑

自主探索

1. 尝试使用三元运算符实现成绩等级判断
2. 搜索了解Dangling Else问题，编写代码验证
3. 尝试实现一个分段函数计算器

参考答案

练习1 参考答案

```
#include <stdio.h>

int main(void) {
    int n;
    printf("请输入整数: ");
    scanf("%d", &n);
    if (n % 2 == 0) {
        printf("偶数\n");
    } else {
        printf("奇数\n");
    }
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    int year;
    printf("请输入年份: ");
    scanf("%d", &year);
    if (year % 400 == 0) {
        printf("闰年\n");
    } else if (year % 100 == 0) {
        printf("平年\n");
    } else if (year % 4 == 0) {
        printf("闰年\n");
    } else {
        printf("平年\n");
    }
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int main(void) {
    float num1, num2;
    char op;
    printf("请输入表达式 (如: 3 + 5) : ");
    scanf("%f %c %f", &num1, &op, &num2);
    if (op == '+') {
        printf("%.2f\n", num1 + num2);
    } else if (op == '-') {
        printf("%.2f\n", num1 - num2);
    } else if (op == '*') {
        printf("%.2f\n", num1 * num2);
    } else if (op == '/') {
        if (num2 != 0) {
            printf("%.2f\n", num1 / num2);
        }
    }
}
```

```

    } else {
        printf("除数不能为0\n");
    }
} else {
    printf("未知运算符\n");
}
return 0;
}

```

挑战题参考答案

```

#include <stdio.h>

int main(void) {
    int x, a, b, c, t;
    printf("输入x: ");
    scanf("%d", &x);
    printf("sign=%d\n", (x > 0) ? 1 : ((x < 0) ? -1 : 0));

    printf("输入三个整数: ");
    scanf("%d %d %d", &a, &b, &c);
    if (a < b) { t = a; a = b; b = t; }
    if (a < c) { t = a; a = c; c = t; }
    if (b < c) { t = b; b = c; c = t; }
    printf("%d %d %d\n", a, b, c);
    return 0;
}

```

Day 5: 流程控制（下）

今日目标：掌握switch语句，学会编写菜单程序

重点知识点讲解

5.1 switch语句基本语法

```

switch (表达式) {
    case 常量1:
        // 语句1
        break;
    case 常量2:
        // 语句2
        break;
    default:
        // 默认语句
}

```

5.2 switch语句的执行流程

1. 计算switch后面表达式的值
2. 匹配对应的case

3. 执行case后面的语句
4. 遇到break跳出switch

5.3 case穿透

当case后面没有break时，会继续执行下一个case的语句：

```
switch (grade) {
    case 'A':
    case 'B':
    case 'C':
        printf("成绩合格");
        break;
    case 'D':
    case 'E':
        printf("成绩不合格");
        break;
}
```

5.4 进阶：switch 的限制与有意穿透

1. `switch` 表达式必须是整型或字符型（不能直接 `switch` 浮点数或字符串）。
2. `case` 标签必须是**常量表达式**，不能是变量。
3. **有意穿透**适合「多种输入同一处理」；无意穿透是 bug。每个 `case` 写完先问自己：要不要 `break`？
4. `default` 不是必须，但强烈建议写上，便于发现非法输入。

知识点实战例题

例题1：用 switch 输出月份天数（不考虑闰年）

题目：输入月份 1~12，输出该月天数；非法月份提示错误。

解题思路：

1. `case 1/3/5/7/8/10/12` 可穿透到同一输出 31
2. `case 4/6/9/11` → 30
3. `case 2` → 28
4. 每个逻辑组末尾 `break`；加 `default`

完整代码：

```
#include <stdio.h>

int main(void) {
    int m;
    printf("输入月份: ");
    scanf("%d", &m);
    switch (m) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            printf("31天\n");
            break;
        case 4: case 6: case 9: case 11:
            printf("30天\n");
            break;
```

```
    case 2:
        printf("28天\n");
        break;
    default:
        printf("月份无效\n");
    }
    return 0;
}
```

要点/易错：有意穿透很好用；忘了 `break` 会掉进下一个 case。

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：星期几查询

- 输入数字1-7
- 使用switch输出对应的星期

练习2：简单计算器（使用switch实现）

练习3：学生成绩管理系统菜单

- 显示菜单选项
- 使用switch处理用户选择（功能可用 printf 占位）

挑战题：增强菜单程序

- 当用户输入非 1 ~ 5 时，提示错误并**继续循环**
- 额外：若 `scanf` 读入失败（例如输入了字母），清空错误状态并提示「请输入数字」

常见错误与调试提示

- **错误：**忘记添加break导致case穿透
 - **解决：**每个case后面都要添加break，除非有意使用case穿透
- **错误：**switch表达式不是整型或字符型
 - **解决：**switch的表达式必须是整型或字符型常量
- **错误：**case后面使用变量
 - **解决：**case必须是常量表达式

自主探索

1. 尝试使用switch语句实现简单的四则运算计算器
2. 搜索了解switch和if-else的性能差异
3. 尝试实现一个简单的命令行菜单系统

参考答案

练习1 参考答案

```
#include <stdio.h>
```



```

int main(void) {
    int d;
    printf("输入1-7: ");
    scanf("%d", &d);
    switch (d) {
        case 1: printf("星期一\n"); break;
        case 2: printf("星期二\n"); break;
        case 3: printf("星期三\n"); break;
        case 4: printf("星期四\n"); break;
        case 5: printf("星期五\n"); break;
        case 6: printf("星期六\n"); break;
        case 7: printf("星期日\n"); break;
        default: printf("输入错误\n");
    }
    return 0;
}

```

练习2 参考答案

```

#include <stdio.h>

int main(void) {
    double a, b;
    char op;
    printf("输入表达式 (如 3 + 5) : ");
    scanf("%lf %c %lf", &a, &op, &b);
    switch (op) {
        case '+': printf("%.2f\n", a + b); break;
        case '-': printf("%.2f\n", a - b); break;
        case '*': printf("%.2f\n", a * b); break;
        case '/':
            if (b != 0) printf("%.2f\n", a / b);
            else printf("除数不能为0\n");
            break;
        default: printf("未知运算符\n");
    }
    return 0;
}

```

练习3 参考答案

```

#include <stdio.h>

int main(void) {
    int choice;
    while (1) {
        printf("\n1.添加 2.查询 3.修改 4.删除 5.退出\n选择: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("执行添加\n"); break;
            case 2: printf("执行查询\n"); break;
            case 3: printf("执行修改\n"); break;

```

```

        case 4: printf("执行删除\n"); break;
        case 5: printf("再见\n"); return 0;
        default: printf("输入错误\n");
    }
}
}

```

挑战题参考答案

思路: scanf 返回值不为 1 时, 用循环读掉非法字符到换行, 再提示重输。

```

#include <stdio.h>

int main(void) {
    int choice;
    int ch;
    while (1) {
        printf("\n1-4功能 5退出: ");
        if (scanf("%d", &choice) != 1) {
            while ((ch = getchar()) != '\n' && ch != EOF) { }
            printf("请输入数字\n");
            continue;
        }
        switch (choice) {
            case 1: case 2: case 3: case 4:
                printf("执行功能%d\n", choice);
                break;
            case 5:
                return 0;
            default:
                printf("输入错误\n");
        }
    }
}

```

Day 6: 循环结构（上）

今日目标: 掌握while和do-while循环

重点知识点讲解

6.1 while循环

```

while (条件) {
    // 循环体
}

```

执行流程: 先判断条件, 条件为真执行循环体, 否则退出循环

6.2 do-while循环

```
do {  
    // 循环体  
} while (条件);
```

执行流程：先执行循环体，再判断条件，至少执行一次

6.3 while vs do-while

特性	while	do-while
执行顺序	先判断后执行	先执行后判断
执行次数	0次或多次	至少1次

6.4 进阶：哨兵循环与累加器

哨兵值 (sentinel)：用特殊输入表示结束，例如输入 结束成绩录入。

```
int score, sum = 0, count = 0;  
printf("输入成绩, -1结束: ");  
while (scanf("%d", &score) == 1 && score != -1) {  
    sum += score;    /* 累加器 */  
    count++;        /* 计数器 */  
    printf("继续输入, -1结束: ");  
}
```

模式记忆：**初始化** → **判断** → **处理** → **更新** → **再判断**。缺了「更新」就容易死循环。

知识点实战例题

例题1：累加 1~n

题目：输入正整数 n，用 while 计算 1+2+...+n。

解题思路：

1. 读入 n，初始化 sum=0, i=1
2. while i<=n: sum+=i; i++
3. 输出 sum

完整代码：

```
#include <stdio.h>

int main(void) {
    int n, i = 1, sum = 0;
    printf("输入n: ");
    scanf("%d", &n);
    while (i <= n) {
        sum += i;
        i++;
    }
    printf("sum=%d\n", sum);
    return 0;
}
```

要点/易错：循环内必须更新 i，否则死循环。

例题2：菜单至少执行一次

题目：用 do-while 实现：显示菜单，输入 0 退出，其它打印“继续”。

解题思路：菜单适合 do-while——先显示再判断是否退出。

完整代码：

```
#include <stdio.h>

int main(void) {
    int c;
    do {
        printf("1.开始 0.退出\n选择: ");
        scanf("%d", &c);
        if (c != 0) printf("继续...\n");
    } while (c != 0);
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：输出1到100

- 使用while循环

练习2：计算1到100的和

- 使用while循环

练习3：输入正数验证

- 使用do-while循环确保输入正数

挑战题：

1. 不断输入整数直到输入 0，输出这些数的和与平均值（注意 count==0）
2. 输入一个正整数，统计它有多少位（例如 12345 → 5）

常见错误与调试提示

- **错误：**无限循环
 - **解决：**确保循环中有更新条件的语句，避免死循环
- **错误：**do-while循环末尾忘记加分号
 - **解决：**do-while循环必须以分号结尾
- **错误：**循环条件错误
 - **解决：**仔细检查条件表达式，确保逻辑正确

自主探索

1. 使用while循环计算阶乘
2. 使用do-while循环实现菜单程序
3. 搜索了解循环的时间复杂度概念

参考答案

练习1 参考答案

```
#include <stdio.h>

int main(void) {
    int i = 1;
    while (i <= 100) {
        printf("%d ", i);
        i++;
    }
    printf("\n");
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    int i = 1, sum = 0;
    while (i <= 100) {
        sum += i;
        i++;
    }
    printf("%d\n", sum);
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int main(void) {
    int num;
    do {
        printf("请输入一个正数: ");
        scanf("%d", &num);
    } while (num <= 0);
    printf("你输入的正数是: %d\n", num);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
    int x, sum = 0, count = 0;
    printf("输入整数, 0结束: ");
    while (scanf("%d", &x) == 1 && x != 0) {
        sum += x;
        count++;
    }
    if (count == 0) printf("没有有效数据\n");
    else printf("和=%d 平均=%.2f\n", sum, (double)sum / count);

    int n, digits = 0;
    printf("输入正整数统计位数: ");
    scanf("%d", &n);
    if (n == 0) digits = 1;
    else {
        while (n > 0) {
            digits++;
            n /= 10;
        }
    }
    printf("位数=%d\n", digits);
    return 0;
}
```

Day 7: 循环结构（下）

今日目标: 掌握for循环和循环嵌套, 理解break/continue

重点知识点讲解

7.1 for循环

```
for (初始化; 条件; 更新) {  
    // 循环体  
}
```

示例：输出1到10

```
for (int i = 1; i <= 10; i++) {  
    printf("%d ", i);  
}
```

7.2 break语句

跳出当前循环：

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;  
    printf("%d ", i);  
}  
// 输出: 1 2 3 4
```

7.3 continue语句

跳过当前迭代，继续下一次循环：

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue;  
    printf("%d ", i);  
}  
// 输出: 1 3 5 7 9
```

7.4 循环嵌套

循环中包含另一个循环：

```
// 打印九九乘法表  
for (int i = 1; i <= 9; i++) {  
    for (int j = 1; j <= i; j++) {  
        printf("%d×%d=%d\t", j, i, i * j);  
    }  
    printf("\n");  
}
```

7.5 进阶：break 作用范围与质数判断优化

- `break` / `continue` 只作用于最近的那一层循环。要跳出多层，通常用标志变量，或把逻辑拆成函数后 `return`。
- 判断质数时，试除到 `sqrt(n)`（即 `i * i <= n`）即可，不必试到 `n-1`。
- `for (int i = 1; i <= n; i++)` 里在 C99 才允许在 `for` 内声明 `i`；若编译报错，把 `int i;` 提到循环外，或在 Dev-C++ 编译选项中开启 C99/GNU。

知识点实战例题

例题1: for 打印偶数

题目: 用 for 打印 1 ~ 20 中的偶数; 遇到 16 用 continue 跳过。

解题思路: i 从 1 到 20; 若 i==16 continue; 若 i%2==0 则打印。

完整代码:

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 1; i <= 20; i++) {
        if (i == 16) continue;
        if (i % 2 == 0) printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

例题2: 嵌套打印矩形

题目: 输入行数 r、列数 c, 打印 r 行 c 列的  矩形。

解题思路: 外层控行, 内层控列, 行末换行。

完整代码:

```
#include <stdio.h>

int main(void) {
    int r, c, i, j;
    printf("行 列: ");
    scanf("%d %d", &r, &c);
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) printf("*");
        printf("\n");
    }
    return 0;
}
```

作业练习

先独立完成, 再到文末「参考答案」对照。

练习1: 计算阶乘

- 输入一个正整数
- 使用for循环计算阶乘

练习2: 打印金字塔

- 使用嵌套循环打印星号金字塔

练习3: 判断质数

- 输入一个正整数
- 判断是否为质数

挑战题：

1. 打印菱形（由空格和 * 组成，输入奇数行高）
2. 完数判断：一个数若等于其所有真因子之和（如 $6=1+2+3$ ），则输出「完数」；找出 2 ~ 1000 内所有完数

常见错误与调试提示

- **错误：**for循环中初始化表达式执行多次
 - **解决：**初始化表达式只执行一次，不要在其中写需要重复执行的代码
- **错误：**循环嵌套导致逻辑混乱
 - **解决：**外层循环控制行数，内层循环控制列数，使用缩进提高可读性
- **错误：**误用break和continue
 - **解决：**break跳出循环，continue跳过当前迭代，注意区别

自主探索

1. 使用循环嵌套打印菱形
2. 使用循环实现斐波那契数列
3. 搜索了解goto语句的使用场景和争议

参考答案

练习1 参考答案

```
#include <stdio.h>

int main(void) {
    int n, i;
    long long fac = 1;
    printf("输入正整数: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) fac *= i;
    printf("%d! = %lld\n", n, fac);
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    int n = 5, i, j, k;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (k = 1; k <= 2 * i - 1; k++) printf("*");
        printf("\n");
    }
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int main(void) {
    int num, i, is_prime = 1;
    printf("请输入一个正整数: ");
    scanf("%d", &num);
    if (num <= 1) is_prime = 0;
    else {
        for (i = 2; i * i <= num; i++) {
            if (num % i == 0) { is_prime = 0; break; }
        }
    }
    printf(is_prime ? "%d是质数\n" : "%d不是质数\n", num);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
    int n, i, j, k;
    printf("输入奇数行高: ");
    scanf("%d", &n);
    int mid = n / 2;
    for (i = 0; i < n; i++) {
        int stars = i <= mid ? (2 * i + 1) : (2 * (n - 1 - i) + 1);
        int spaces = (n - stars) / 2;
        for (j = 0; j < spaces; j++) printf(" ");
        for (k = 0; k < stars; k++) printf("*");
        printf("\n");
    }

    int m, s, d;
    for (m = 2; m <= 1000; m++) {
        s = 0;
        for (d = 1; d < m; d++) if (m % d == 0) s += d;
        if (s == m) printf("%d ", m);
    }
}
```

```
}  
printf("\n");  
return 0;  
}
```

Day 8: 数组

今日目标：理解数组的概念，学会使用数组存储和处理批量数据

重点知识点讲解

8.1 数组的声明与初始化

```
// 声明并初始化  
int scores[5] = {85, 90, 78, 92, 88};  
  
// 省略大小（由初始化列表决定）  
int arr[] = {1, 2, 3, 4, 5};  
  
// 部分初始化（未初始化的元素默认为0）  
int arr[5] = {1, 2, 3};  
  
// 全部初始化为0  
int arr[5] = {0};
```

8.2 数组的访问

数组通过索引访问，索引从0开始：

```
int arr[5] = {10, 20, 30, 40, 50};  
arr[0] = 100; // 修改第一个元素  
printf("%d", arr[2]); // 输出第三个元素：30
```

8.3 数组的遍历

```
int arr[] = {1, 2, 3, 4, 5};  
int len = sizeof(arr) / sizeof(arr[0]);  
  
for (int i = 0; i < len; i++) {  
    printf("%d ", arr[i]);  
}
```

8.4 数组作为函数参数

```
void printArray(int arr[], int len) {
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
}
```

8.5 进阶：越界、sizeof 失效与二维数组

1. **越界是未定义行为**：`arr[5]` 在长度为 5 的数组上非法，可能当时“看起来正常”，实则埋雷。
2. **函数里不要写 `sizeof(arr)/sizeof(arr[0])` 求长度**：参数 `arr` 已退化成指针，`sizeof(arr)` 往往是指针大小。长度必须单独传入。
3. **二维数组入门**：

```
int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d\n", a[1][2]); /* 6 */
```

内存上按行优先连续存放，这为后面理解指针打下基础。

知识点实战例题

例题1：数组求和

题目：定义数组 `{1,3,5,7,9}`，求元素之和。

解题思路：用 `sizeof` 求长度，for 累加。

完整代码：

```
#include <stdio.h>

int main(void) {
    int a[] = {1, 3, 5, 7, 9};
    int len = sizeof(a) / sizeof(a[0]);
    int i, sum = 0;
    for (i = 0; i < len; i++) sum += a[i];
    printf("sum=%d\n", sum);
    return 0;
}
```

例题2：查找元素

题目：在数组中查找目标值，找到打印下标，否则打印 -1。

解题思路：顺序扫描；找到即 break 并记录下标。

完整代码：

```
#include <stdio.h>

int main(void) {
    int a[] = {4, 2, 9, 7, 1};
    int target = 7, i, idx = -1;
    int len = sizeof(a) / sizeof(a[0]);
    for (i = 0; i < len; i++) {
        if (a[i] == target) { idx = i; break; }
    }
    printf("%d\n", idx);
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：数组求最大值、最小值、平均值

练习2：冒泡排序

- 对数组进行升序排序

练习3：数组逆序

- 将数组元素逆序存放

挑战题：

1. 在有序数组中插入一个新元素，保持有序（注意数组容量）
2. 实现选择排序，并与冒泡排序对比交换次数（可加计数器）

常见错误与调试提示

- **错误：**数组越界访问
 - **解决：**确保索引在有效范围内，避免访问不存在的元素
- **错误：**数组作为函数参数时丢失大小信息
 - **解决：**数组作为参数时会退化为指针，需要额外传递长度参数
- **错误：**使用 `==` 比较数组
 - **解决：**数组名是指针，`==` 比较的是地址，需要逐元素比较

自主探索

1. 实现选择排序算法
2. 实现二分查找算法
3. 搜索了解二维数组的内存布局

参考答案

练习1 参考答案

```
#include <stdio.h>
```

```

int main(void) {
    int a[5], i, max, min;
    double avg = 0;
    printf("输入5个整数: ");
    for (i = 0; i < 5; i++) scanf("%d", &a[i]);
    max = min = a[0];
    for (i = 0; i < 5; i++) {
        if (a[i] > max) max = a[i];
        if (a[i] < min) min = a[i];
        avg += a[i];
    }
    avg /= 5;
    printf("max=%d min=%d avg=%.2f\n", max, min, avg);
    return 0;
}

```

练习2 参考答案

```

#include <stdio.h>

void bubbleSort(int arr[], int len) {
    int i, j, t;
    for (i = 0; i < len - 1; i++)
        for (j = 0; j < len - 1 - i; j++)
            if (arr[j] > arr[j + 1]) {
                t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
}

int main(void) {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int len = sizeof(arr) / sizeof(arr[0]), i;
    bubbleSort(arr, len);
    for (i = 0; i < len; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}

```

练习3 参考答案

```

#include <stdio.h>

int main(void) {
    int a[] = {1, 2, 3, 4, 5};
    int len = 5, i, t;
    for (i = 0; i < len / 2; i++) {
        t = a[i]; a[i] = a[len - 1 - i]; a[len - 1 - i] = t;
    }
    for (i = 0; i < len; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}

```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
    int a[10] = {1, 3, 5, 7, 9}, n = 5, x = 6, i, j;
    /* 有序插入 */
    for (i = n - 1; i >= 0 && a[i] > x; i--) a[i + 1] = a[i];
    a[i + 1] = x; n++;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");

    /* 选择排序 */
    int b[] = {5, 2, 8, 1, 9}, m = 5, min, t;
    for (i = 0; i < m - 1; i++) {
        min = i;
        for (j = i + 1; j < m; j++) if (b[j] < b[min]) min = j;
        t = b[i]; b[i] = b[min]; b[min] = t;
    }
    for (i = 0; i < m; i++) printf("%d ", b[i]);
    printf("\n");
    return 0;
}
```

Day 9: 字符串

今日目标: 理解字符串的概念, 学会使用字符串处理函数

重点知识点讲解

9.1 字符串的定义

在C语言中, 字符串是以 `'\0'` 结尾的字符数组:

```
char str1[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
char str2[] = "Hello"; // 推荐方式
```

9.2 常用字符串函数

函数	功能
<code>strlen(str)</code>	计算字符串长度
<code>strcpy(dest, src)</code>	字符串复制
<code>strcat(dest, src)</code>	字符串拼接
<code>strcmp(str1, str2)</code>	字符串比较

9.3 字符串输入输出

```
char name[50];
scanf("%s", name);           // 输入（遇到空格停止）
fgets(name, sizeof(name), stdin); // 输入整行
printf("%s", name);         // 输出
```

9.4 进阶：'\0'、危险的 gets 与更安全的读入

1. 字符串长度 `strlen` 不含 `'\0'`，但数组空间必须为 `'\0'` 留位置：`"Hi"` 至少要 `char s[3]`。
2. 永远不要使用 `gets`：无法限制长度，极易缓冲区溢出。教材若出现，请改用 `fgets`。
3. `fgets` 会读入换行符，常用处理：

```
char line[100];
if (fgets(line, sizeof(line), stdin)) {
    size_t n = strlen(line);
    if (n > 0 && line[n - 1] == '\n') {
        line[n - 1] = '\0';
    }
}
```

4. 需要从字符串里解析数字时，可了解 `sscanf(line, "%d", &x);`。

知识点实战例题

例题1：手写统计长度

题目：不调用 `strlen`，统计字符串长度。

解题思路：从下标 0 走到 `\0` 为止。

完整代码：

```
#include <stdio.h>

int main(void) {
    char s[] = "Hello";
    int i = 0;
    while (s[i] != '\0') i++;
    printf("len=%d\n", i);
    return 0;
}
```

例题2：fgets 读一行并去掉换行

题目：读入一行文本（可含空格），去掉末尾 `\n` 后输出。

解题思路：`fgets` → 找末尾换行改成 `\0`。

完整代码：


```
#include <stdio.h>
#include <string.h>

int main(void) {
    char line[100];
    printf("输入一行: ");
    if (fgets(line, sizeof(line), stdin)) {
        size_t n = strlen(line);
        if (n > 0 && line[n - 1] == '\n') line[n - 1] = '\0';
        printf("[%s]\n", line);
    }
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：字符串长度

- 自己实现strlen函数

练习2：字符串反转

练习3：统计字符出现次数

- 输入字符串和目标字符
- 统计该字符在字符串中出现的次数

挑战题：手写 `my_strcmp`（按字典序比较，返回负/零/正），再用它判断用户输入的密码是否等于 `"secret"`

常见错误与调试提示

- **错误：**字符串忘记添加 `'\0'` 结尾符
 - **解决：**字符串必须以 `'\0'` 结尾，否则会导致未定义行为
- **错误：**使用 `==` 比较字符串
 - **解决：**必须使用 `strcmp` 函数比较字符串内容
- **错误：**`strcpy` 导致缓冲区溢出
 - **解决：**确保目标数组足够大，或使用 `strncpy`

自主探索

1. 自己实现strcpy和strcmp函数
2. 实现字符串拼接函数
3. 搜索了解字符串的内存存储方式

参考答案

练习1 参考答案

```

#include <stdio.h>

int my_strlen(const char *str) {
    int len = 0;
    while (str[len] != '\0') len++;
    return len;
}

int main(void) {
    char str[] = "Hello, World!";
    printf("长度: %d\n", my_strlen(str));
    return 0;
}

```

练习2 参考答案

```

#include <stdio.h>

int main(void) {
    char s[100];
    int i, j;
    char t;
    printf("输入字符串: ");
    scanf("%s", s);
    for (i = 0; s[i] != '\0'; i++) { }
    for (j = 0; j < i / 2; j++) {
        t = s[j]; s[j] = s[i - 1 - j]; s[i - 1 - j] = t;
    }
    printf("%s\n", s);
    return 0;
}

```

练习3 参考答案

```

#include <stdio.h>

int main(void) {
    char s[100], ch;
    int i, cnt = 0;
    printf("字符串: ");
    scanf("%s", s);
    printf("目标字符: ");
    scanf(" %c", &ch);
    for (i = 0; s[i] != '\0'; i++) if (s[i] == ch) cnt++;
    printf("出现%d次\n", cnt);
    return 0;
}

```

挑战题参考答案

```

#include <stdio.h>

```

```
int my_strcmp(const char *a, const char *b) {
    while (*a && (*a == *b)) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;
}

int main(void) {
    char pwd[50];
    printf("密码: ");
    scanf("%s", pwd);
    if (my_strcmp(pwd, "secret") == 0) printf("正确\n");
    else printf("错误\n");
    return 0;
}
```

Day 10: 函数

今日目标：掌握函数的定义与调用，理解参数传递和递归

重点知识点讲解

10.1 函数定义

```
返回类型 函数名(参数列表) {
    // 函数体
    return 返回值;
}
```

10.2 参数传递方式

值传递：函数对参数的修改不影响实参

```
void increment(int x) {
    x++;
}
```

地址传递：函数对参数的修改会影响实参

```
void increment(int *x) {
    (*x)++;
}
```

10.3 函数原型

函数原型是函数的声明，用于类型检查：

```
int add(int a, int b); // 函数原型

int main() {
    add(3, 5);
    return 0;
}

int add(int a, int b) { // 函数定义
    return a + b;
}
```

10.4 递归函数

函数调用自身，需要终止条件：

```
int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

10.5 进阶：形参/实参、值传递局限、递归 vs 循环

1. **实参**是调用时传入的值；**形参**是函数定义中的变量。值传递时形参是实参的副本。
2. 想在函数内修改外部变量，必须传**地址**（指针）——这是后面学指针的动机之一。
3. 能用循环写清的，不必强行递归。递归适合分治、树形结构；注意栈深度，`n` 很大时阶乘递归可能栈溢出，循环更稳。

知识点实战例题

例题1：函数求两数之和

题目：编写 `add(a,b)`，在 `main` 中调用并打印。

解题思路：先声明原型（或定义在 `main` 前）→ 调用 → `return`。

完整代码：

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main(void) {
    printf("%d\n", add(3, 5));
    return 0;
}
```

例题2：递归求阶乘（理解调用栈）

题目：写 `fac(n)` 递归版，计算 `5!`。

解题思路：边界 $n \leq 1$ 返回 1；否则 $n * \text{fac}(n-1)$ 。

完整代码：

```
#include <stdio.h>

long long fac(int n) {
    if (n <= 1) return 1;
    return n * fac(n - 1);
}

int main(void) {
    printf("%lld\n", fac(5));
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：计算数组的最大值（使用函数）

练习2：递归计算幂

练习3：二分查找

- 在有序数组中查找目标值

挑战题：

- 写 `gcd(a, b)`：递归版与循环版各一（欧几里得算法）
- 了解汉诺塔：写出 `hanoi(n, from, to, via)` 的递归框架，并对 `n=3` 跟踪打印步骤（理解优先，不强求一次写对）

常见错误与调试提示

- 错误：**函数参数类型不匹配
 - 解决：**确保函数调用时参数类型和数量与定义一致
- 错误：**递归没有终止条件
 - 解决：**递归函数必须有明确的终止条件，否则会导致栈溢出
- 错误：**函数返回值类型不匹配
 - 解决：**确保返回值类型与函数声明一致

自主探索

- 使用递归实现斐波那契数列
- 搜索了解尾递归的概念
- 尝试将递归函数转换为迭代实现

参考答案

练习1 参考答案

```
#include <stdio.h>

int max_of(int arr[], int len) {
    int i, m = arr[0];
    for (i = 1; i < len; i++) if (arr[i] > m) m = arr[i];
    return m;
}

int main(void) {
    int a[] = {3, 9, 2, 7};
    printf("%d\n", max_of(a, 4));
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

long long power(int base, int exp) {
    if (exp == 0) return 1;
    return base * power(base, exp - 1);
}

int main(void) {
    printf("%lld\n", power(2, 10));
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int binarySearch(int arr[], int len, int target) {
    int left = 0, right = len - 1, mid;
    while (left <= right) {
        mid = (left + right) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

int main(void) {
    int arr[] = {1, 3, 5, 7, 9, 11, 13};
    int idx = binarySearch(arr, 7, 7);
    printf("%d\n", idx);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int gcd_rec(int a, int b) {
    if (b == 0) return a;
    return gcd_rec(b, a % b);
}

int gcd_loop(int a, int b) {
    int t;
    while (b != 0) { t = a % b; a = b; b = t; }
    return a;
}

void hanoi(int n, char from, char to, char via) {
    if (n == 1) {
        printf("%c -> %c\n", from, to);
        return;
    }
    hanoi(n - 1, from, via, to);
    printf("%c -> %c\n", from, to);
    hanoi(n - 1, via, to, from);
}

int main(void) {
    printf("gcd=%d / %d\n", gcd_rec(48, 18), gcd_loop(48, 18));
    hanoi(3, 'A', 'C', 'B');
    return 0;
}
```

Day 11: 指针 (上)

今日目标：理解指针的概念，掌握指针与数组的关系

重点知识点讲解

11.1 指针的概念

指针是一个变量，存储的是另一个变量的内存地址：

```
int num = 42;
int *ptr = &num; // ptr指向num
```

11.2 指针运算符

运算符	名称	作用
&	取地址运算符	获取变量的内存地址
*	解引用运算符	获取指针指向的值

11.3 指针与数组

数组名本质上是指向数组第一个元素的指针：

```
int arr[] = {10, 20, 30};
arr[0] == *arr;    // true
arr[1] == *(arr+1); // true
```

11.4 使用指针遍历数组

```
int arr[] = {1, 2, 3, 4, 5};
int len = sizeof(arr) / sizeof(arr[0]);

for (int *p = arr; p < arr + len; p++) {
    printf("%d ", *p);
}
```

11.5 进阶：指针类型、const 与指针算术

- 指针有类型：`int *` 一次移动 `sizeof(int)` 字节；类型决定解引用时如何解释内存。
- `const` 两种常见写法（含义不同）：
 - `const int *p`：不能通过 `p` 改所指向的值
 - `int *const p`：`p` 本身不能改指向
- `p + 1` 不是地址数字加 1，而是指向下一个同类型元素。
- 数组名多数场合会退化成指针，但数组名不是“可赋值的指针变量”：不能写 `arr = ...`。

知识点实战例题

例题1：指针修改变量

题目：通过指针把变量从 10 改成 20。

解题思路：`p=&x; *p=20;`

完整代码：


```
#include <stdio.h>

int main(void) {
    int x = 10;
    int *p = &x;
    *p = 20;
    printf("%d\n", x);
    return 0;
}
```

例题2：指针遍历数组

题目：用指针打印数组全部元素。

解题思路： `p` 从 `arr` 走到 `arr+len`。

完整代码：

```
#include <stdio.h>

int main(void) {
    int arr[] = {1, 2, 3, 4};
    int *p;
    for (p = arr; p < arr + 4; p++) printf("%d ", *p);
    printf("\n");
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：使用指针实现数组逆序

练习2：使用指针交换两个整数

练习3：字符串复制（使用指针实现）

挑战题：用指针实现在数组中查找目标值，返回指向该元素的指针；找不到返回 `NULL`，并在 `main` 中安全处理

常见错误与调试提示

- **错误：**空指针解引用
 - **解决：**使用指针前检查是否为 `NULL`
- **错误：**野指针（未初始化的指针）
 - **解决：**初始化指针，避免使用未初始化的指针
- **错误：**指针运算越界
 - **解决：**确保指针操作在有效范围内

自主探索

1. 使用指针实现选择排序

2. 搜索了解指针数组和数组指针的区别
3. 尝试使用指针实现字符串反转

参考答案

练习1 参考答案

```
#include <stdio.h>

void reverse(int *a, int len) {
    int *l = a, *r = a + len - 1, t;
    while (l < r) {
        t = *l; *l = *r; *r = t;
        l++; r--;
    }
}

int main(void) {
    int a[] = {1, 2, 3, 4, 5}, i;
    reverse(a, 5);
    for (i = 0; i < 5; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

int main(void) {
    int x = 3, y = 5;
    swap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

void my_strcpy(char *dest, const char *src) {
    while (*src != '\0') {
        *dest = *src;
        dest++; src++;
    }
    *dest = '\0';
}
```

```
int main(void) {
    char src[] = "Hello, World!";
    char dest[20];
    my_strcpy(dest, src);
    printf("%s\n", dest);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int *find(int *a, int len, int target) {
    int i;
    for (i = 0; i < len; i++)
        if (a[i] == target) return &a[i];
    return NULL;
}

int main(void) {
    int a[] = {4, 2, 9, 7};
    int *p = find(a, 4, 9);
    if (p) printf("found %d\n", *p);
    else printf("NULL\n");
    return 0;
}
```

Day 12: 指针（下）

今日目标：掌握动态内存分配，理解指针的高级应用

重点知识点讲解

12.1 动态内存分配函数

函数	功能
<code>malloc(size)</code>	分配指定大小的内存
<code>calloc(n, size)</code>	分配n个size大小的内存并初始化为0
<code>realloc(ptr, size)</code>	重新分配内存大小
<code>free(ptr)</code>	释放已分配的内存

12.2 动态内存分配示例

```
#include <stdlib.h>
```

```
// 分配单个整数
int *p = (int *)malloc(sizeof(int));
*p = 42;

// 分配数组
int *arr = (int *)malloc(5 * sizeof(int));
for (int i = 0; i < 5; i++) {
    arr[i] = i * 10;
}

// 释放内存
free(p);
free(arr);
```

12.3 常见指针错误

- **空指针解引用**: `int *p = NULL; *p = 10;`
- **野指针**: 未初始化的指针
- **内存泄漏**: 忘记释放动态分配的内存

12.4 进阶: malloc 失败、二次释放与 realloc

```
int *arr = (int *)malloc(n * sizeof(int));
if (arr == NULL) {
    printf("内存分配失败\n");
    return 1;
}
/* 使用 arr ... */
free(arr);
arr = NULL; /* 防止悬空指针被再次使用 */
```

要点:

1. 每次 `malloc` / `calloc` 都要检查是否为 `NULL`。
2. 谁分配谁释放; 同一块内存不要 `free` 两次。
3. `realloc` 可能返回新地址: 应先用临时指针接住, 成功后再赋回, 失败时旧块仍有效。
4. 释放后把指针置 `NULL`, 是简单有效的防护习惯。

知识点实战例题

例题1: malloc 分配单个 int

题目: 动态分配一个 int, 赋值为 42, 打印后释放。

解题思路: malloc → 判空 → 使用 → free → 置 NULL。

完整代码:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = (int *)malloc(sizeof(int));
    if (!p) return 1;
    *p = 42;
    printf("%d\n", *p);
    free(p);
    p = NULL;
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：动态数组实现

- 创建动态数组，支持添加元素和自动扩容

练习2：动态二维数组

练习3：指针实现链表（单链表基本操作）

挑战题：改造上面的 `addElement`：用临时指针接收 `realloc` 结果；若失败则打印错误且不丢失原数据；在 `createDynamicArray` 中检查每一次分配是否成功

常见错误与调试提示

- **错误：**忘记释放动态分配的内存
 - **解决：**动态分配的内存必须使用`free`释放
- **错误：**重复释放内存
 - **解决：**确保每个`malloc`只对应一个`free`
- **错误：**使用已释放的内存
 - **解决：**释放后将指针置为`NULL`，使用前检查

自主探索

1. 实现一个简单的内存池
2. 搜索了解智能指针的概念
3. 尝试实现双向链表

参考答案

练习1 参考答案（含安全 `realloc` 思路）

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
```

```

    int size;
    int capacity;
} DynamicArray;

DynamicArray *createDynamicArray(int cap) {
    DynamicArray *arr = (DynamicArray *)malloc(sizeof(DynamicArray));
    if (!arr) return NULL;
    arr->data = (int *)malloc(cap * sizeof(int));
    if (!arr->data) { free(arr); return NULL; }
    arr->size = 0;
    arr->capacity = cap;
    return arr;
}

int addElement(DynamicArray *arr, int element) {
    if (arr->size >= arr->capacity) {
        int newcap = arr->capacity * 2;
        int *tmp = (int *)realloc(arr->data, newcap * sizeof(int));
        if (!tmp) return 0;
        arr->data = tmp;
        arr->capacity = newcap;
    }
    arr->data[arr->size++] = element;
    return 1;
}

void destroyDynamicArray(DynamicArray *arr) {
    free(arr->data);
    free(arr);
}

int main(void) {
    DynamicArray *arr = createDynamicArray(2);
    int i;
    addElement(arr, 10);
    addElement(arr, 20);
    addElement(arr, 30);
    for (i = 0; i < arr->size; i++) printf("%d ", arr->data[i]);
    printf("\n");
    destroyDynamicArray(arr);
    return 0;
}

```

练习2 参考答案

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int r = 2, c = 3, i, j;
    int **a = (int **)malloc(r * sizeof(int *));
    for (i = 0; i < r; i++) a[i] = (int *)malloc(c * sizeof(int));
    for (i = 0; i < r; i++)

```

```

    for (j = 0; j < c; j++) a[i][j] = i * c + j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) printf("%d ", a[i][j]);
        printf("\n");
    }
    for (i = 0; i < r; i++) free(a[i]);
    free(a);
    return 0;
}

```

练习3 参考答案

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main(void) {
    Node *head = NULL, *p;
    int vals[3] = {1, 2, 3}, i;
    for (i = 2; i >= 0; i--) {
        p = (Node *)malloc(sizeof(Node));
        p->data = vals[i];
        p->next = head;
        head = p;
    }
    for (p = head; p; p = p->next) printf("%d ", p->data);
    printf("\n");
    while (head) {
        p = head; head = head->next; free(p);
    }
    return 0;
}

```

挑战题参考答案：见练习1 中的 `addElement`（临时指针 + 失败不丢数据）。

Day 13: 结构体

今日目标：掌握结构体的定义与使用，学会创建复杂数据结构

重点知识点讲解

13.1 结构体定义

```
struct Person {
    char name[50];
    int age;
    float height;
};
```

13.2 结构体初始化

```
struct Person p1 = {"张三", 25, 1.75f};

struct Person p2 = {
    .name = "李四",
    .age = 30,
    .height = 1.80f
};
```

13.3 结构体访问

```
struct Person p = {"张三", 25, 1.75f};

// 使用.运算符
printf("%s", p.name);

// 使用指针和->运算符
struct Person *ptr = &p;
printf("%d", ptr->age);
```

13.4 结构体数组

```
struct Student students[3] = {
    {101, "张三", 95.5f},
    {102, "李四", 88.0f},
    {103, "王五", 92.3f}
};
```

13.5 typedef的使用

```
typedef struct {
    char name[50];
    int age;
} Person;

Person p = {"张三", 25};
```

13.6 进阶：结构体对齐与传参方式

1. **对齐（了解）**：编译器可能在成员之间插入填充字节，故 `sizeof(struct)` 不一定等于各成员大小之和。可用 `printf("%zu\n", sizeof(Person));` 观察。
2. **传结构体 vs 传指针**：
 - 传结构体：复制整份数据，适合小结构

- 传指针：只传地址，适合大结构或需要在函数内修改原对象

3. 链表节点是结构体 + 指针的经典组合：

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

知识点实战例题

例题1：结构体表示点并计算

题目：定义 `Point{x,y}`，输入两点，输出曼哈顿距离 `|x1-x2|+|y1-y2|`。

解题思路：定义结构体 → 读入 → 计算绝对值差之和。

完整代码：

```
#include <stdio.h>  
#include <stdlib.h>  
  
typedef struct {  
    int x, y;  
} Point;  
  
int main(void) {  
    Point a, b;  
    scanf("%d %d %d %d", &a.x, &a.y, &b.x, &b.y);  
    printf("%d\n", abs(a.x - b.x) + abs(a.y - b.y));  
    return 0;  
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：学生信息管理系统

- 使用结构体数组存储学生信息
- 实现添加、查询功能

练习2：日期结构体

- 定义日期结构体
- 实现日期比较功能

练习3：通讯录管理

- 定义联系人结构体
- 实现添加、查找、删除功能

挑战题：

1. 对学生数组按成绩降序排序（冒泡即可）
2. 用 `Node` 实现：头插法插入 3 个节点并遍历打印（为后续数据结构打基础）

常见错误与调试提示

- **错误：** 结构体成员访问错误
 - **解决：** 使用 `.` 访问结构体变量，使用 `->` 访问结构体指针
- **错误：** 结构体数组越界
 - **解决：** 确保索引在有效范围内
- **错误：** 结构体赋值错误
 - **解决：** 同类型结构体可以直接赋值，不同类型需要逐成员赋值

自主探索

1. 实现二叉树节点结构
2. 使用共用体实现类型转换
3. 搜索了解位域的使用场景

参考答案

练习1 参考答案（精简版添加+显示）

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int id;
    char name[50];
    float score;
} Student;

int main(void) {
    Student s[MAX];
    int n = 0, op, i, id;
    do {
        printf("1加2查3列0退: ");
        scanf("%d", &op);
        if (op == 1 && n < MAX) {
            scanf("%d %s %f", &s[n].id, s[n].name, &s[n].score);
            n++;
        } else if (op == 2) {
            scanf("%d", &id);
            for (i = 0; i < n; i++)
                if (s[i].id == id) printf("%s %.1f\n", s[i].name, s[i].score);
        } else if (op == 3) {
            for (i = 0; i < n; i++)
                printf("%d %s %.1f\n", s[i].id, s[i].name, s[i].score);
        }
    } while (op != 0);
    return 0;
}
```

练习2 参考答案

```

#include <stdio.h>

typedef struct { int y, m, d; } Date;

int cmp(Date a, Date b) {
    if (a.y != b.y) return a.y - b.y;
    if (a.m != b.m) return a.m - b.m;
    return a.d - b.d;
}

int main(void) {
    Date a = {2026, 7, 26}, b = {2026, 1, 1};
    int c = cmp(a, b);
    if (c > 0) printf("a较晚\n");
    else if (c < 0) printf("a较早\n");
    else printf("相同\n");
    return 0;
}

```

练习3 参考答案 (通讯录核心)

```

#include <stdio.h>
#include <string.h>

typedef struct {
    char name[30];
    char phone[20];
} Contact;

int main(void) {
    Contact c[50];
    int n = 0, op, i;
    char key[30];
    do {
        printf("1加2查3删0退: ");
        scanf("%d", &op);
        if (op == 1) {
            scanf("%s %s", c[n].name, c[n].phone); n++;
        } else if (op == 2) {
            scanf("%s", key);
            for (i = 0; i < n; i++)
                if (strcmp(c[i].name, key) == 0) printf("%s\n", c[i].phone);
        } else if (op == 3) {
            scanf("%s", key);
            for (i = 0; i < n; i++)
                if (strcmp(c[i].name, key) == 0) {
                    c[i] = c[n - 1]; n--; break;
                }
        }
    } while (op != 0);
    return 0;
}

```

挑战题参考答案

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id; char name[20]; float score;
} Stu;

typedef struct Node {
    int data; struct Node *next;
} Node;

int main(void) {
    Stu s[3] = {{1, "A", 90}, {2, "B", 70}, {3, "C", 85}};
    int i, j; Stu t;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2 - i; j++)
            if (s[j].score < s[j+1].score) { t = s[j]; s[j] = s[j+1]; s[j+1] = t; }
    for (i = 0; i < 3; i++) printf("%s %.0f\n", s[i].name, s[i].score);

    Node *head = NULL, *p;
    int v;
    for (v = 1; v <= 3; v++) {
        p = (Node *)malloc(sizeof(Node));
        p->data = v; p->next = head; head = p;
    }
    for (p = head; p; p = p->next) printf("%d ", p->data);
    printf("\n");
    return 0;
}
```

Day 14: 综合实战

今日目标：完成三个实战项目，综合运用所学知识

综合项目实施思路

项目1 简易计算器 — 实现思路

1. 用死循环打印菜单
2. `switch` 分支处理加减乘除/退出
3. 除法前判断除数是否为 0
4. (加分) 用变量 `last` 保存上次结果，支持连续运算

项目2 学生成绩管理 — 实现思路

1. 定义 `Student` 结构体数组 + `count`

2. 菜单驱动：增删改查、列表、平均分
3. 查找用 for + 学号比较；删除用“末尾覆盖”
4. （加分）按成绩冒泡排序后显示

项目3 图书管理 — 实现思路

1. 定义 `Book`（编号、书名、作者、库存）
2. 借阅前检查库存>0；归还库存+1
3. 查询可按编号；加分可用书名关键字（循环/`strstr`）

实战项目1：简易计算器

功能要求：

1. **基本运算**：支持加、减、乘、除四则运算
2. **连续计算**：支持连续输入多个运算
3. **退出功能**：输入特定命令退出程序
4. **菜单界面**：清晰的操作菜单

输入输出示例：

```
===== 简易计算器 =====
```

1. 加法
2. 减法
3. 乘法
4. 除法
5. 退出

```
请选择操作(1-5): 1
```

```
请输入第一个数: 10
```

```
请输入第二个数: 5
```

```
计算结果: 10 + 5 = 15
```

```
===== 简易计算器 =====
```

1. 加法
2. 减法
3. 乘法
4. 除法
5. 退出

```
请选择操作(1-5): 5
```

```
感谢使用计算器！
```

技术要点：

- switch语句处理运算选择
- 循环实现连续计算
- 错误处理（如除数为0）

加分项（挑战）：

- 支持“上一次结果继续运算”（例如先算出 15，再输入 `* 2` 得到 30）
- 或增加 `%`（整数取余）运算，并处理好类型

自我评估检查清单：

- ☐ 能否正确实现四则运算

- ☐ 能否处理除数为0的情况
- ☐ 能否实现连续计算
- ☐ 界面是否友好清晰
- ☐ （加分）是否完成连续运算或取余

实战项目2：学生成绩管理系统

功能要求：

1. **添加学生**：输入学号、姓名、成绩
2. **查询学生**：根据学号或姓名查询
3. **修改成绩**：修改指定学生的成绩
4. **删除学生**：删除指定学生
5. **显示所有学生**：按表格形式显示
6. **计算平均分**：计算所有学生的平均分

输入输出示例：

===== 学生成绩管理系统 =====

1. 添加学生
2. 查询学生
3. 修改成绩
4. 删除学生
5. 显示所有学生
6. 计算平均分
7. 退出

请选择操作(1-7)：1

请输入学生ID：101

请输入学生姓名：张三

请输入学生成绩：95

学生添加成功！

===== 学生成绩管理系统 =====

1. 添加学生
2. 查询学生
3. 修改成绩
4. 删除学生
5. 显示所有学生
6. 计算平均分
7. 退出

请选择操作(1-7)：5

学生列表：

ID	姓名	成绩
101	张三	95.0

===== 学生成绩管理系统 =====

1. 添加学生
2. 查询学生
3. 修改成绩
4. 删除学生

```
5. 显示所有学生
6. 计算平均分
7. 退出
请选择操作(1-7): 7
感谢使用学生成绩管理系统!
```

技术要点:

- 结构体数组存储学生信息
- 函数封装各个功能
- 循环菜单

加分项 (挑战):

- 增加「按成绩排序显示」功能 (升序或降序)
- 查询时同时支持按学号、按姓名 (姓名可用 `strcmp`)

自我评估检查清单:

- ☐ 能否正确添加学生信息
- ☐ 能否根据学号查询学生
- ☐ 能否修改和删除学生信息
- ☐ 能否正确计算平均分
- ☐ 程序是否健壮 (处理边界情况)
- ☐ (加分) 是否完成排序或双条件查询

实战项目3: 图书管理系统

功能要求:

1. **添加图书**: 输入编号、书名、作者、库存
2. **查询图书**: 根据编号或书名查询
3. **借阅图书**: 借阅指定图书 (库存减1)
4. **归还图书**: 归还指定图书 (库存加1)
5. **显示所有图书**: 按表格形式显示

输入输出示例:

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 1
请输入图书编号: 1001
请输入书名: C语言程序设计
请输入作者: 谭浩强
请输入库存: 10
图书添加成功!
```

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 3
请输入要借阅的图书编号: 1001
借阅成功! 当前库存: 9
```

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 5
```

图书列表:

编号	书名	作者	库存
1001	C语言程序设计	谭浩强	9

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 6
感谢使用图书管理系统!
```

技术要点:

- 结构体数组存储图书信息
- 函数实现各个操作
- 完整的菜单系统

加分项 (挑战) :

- 库存为 0 时借阅应明确提示「无库存」, 不能变成负数
- 增加「按书名模糊查找」: 书名包含某关键字即命中 (可用循环比较字符, 或了解 `strstr`)

自我评估检查清单:

- ☐ 能否正确添加图书信息
- ☐ 能否根据编号查询图书
- ☐ 借阅功能是否正确 (库存减1)
- ☐ 归还功能是否正确 (库存加1)

- ☐ 能否处理库存为0的情况
- ☐ 程序是否有良好的用户体验
- ☐ （加分）无库存提示或书名查找是否完成

参考答案（核心模块）

项目1 参考实现

```
#include <stdio.h>

int main(void) {
    int op;
    double a, b, last = 0;
    int has_last = 0;
    while (1) {
        printf("1+ 2- 3* 4/ 5连续用上次 0退: ");
        scanf("%d", &op);
        if (op == 0) break;
        if (op == 5) {
            if (!has_last) { printf("尚无结果\n"); continue; }
            printf("上次=%.2f, 输入运算符和数 (如 * 2) : ", last);
            {
                char o; double x;
                scanf(" %c %lf", &o, &x);
                if (o == '+') last += x;
                else if (o == '-') last -= x;
                else if (o == '*') last *= x;
                else if (o == '/' && x != 0) last /= x;
                else { printf("错误\n"); continue; }
                printf("结果=%.2f\n", last);
            }
            continue;
        }
        printf("两数: ");
        scanf("%lf %lf", &a, &b);
        switch (op) {
            case 1: last = a + b; break;
            case 2: last = a - b; break;
            case 3: last = a * b; break;
            case 4:
                if (b == 0) { printf("除数0\n"); continue; }
                last = a / b; break;
            default: printf("无效\n"); continue;
        }
        has_last = 1;
        printf("结果=%.2f\n", last);
    }
    return 0;
}
```

项目2 参考实现（核心）

```

#include <stdio.h>
#include <string.h>

#define MAX 100
typedef struct { int id; char name[50]; float score; } Student;

int main(void) {
    Student s[MAX];
    int n = 0, op, i, id;
    float sum;
    while (1) {
        printf("1加2查3改4删5列6均7排序0退: ");
        scanf("%d", &op);
        if (op == 0) break;
        if (op == 1 && n < MAX) {
            scanf("%d %s %f", &s[n].id, s[n].name, &s[n].score); n++;
        } else if (op == 2) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (s[i].id == id)
                printf("%s %.1f\n", s[i].name, s[i].score);
        } else if (op == 3) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (s[i].id == id) scanf("%f", &s[i].score);
        } else if (op == 4) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (s[i].id == id) { s[i] = s[n-1]; n--; break; }
        } else if (op == 5) {
            for (i = 0; i < n; i++) printf("%d %s %.1f\n", s[i].id, s[i].name, s[i].score);
        } else if (op == 6) {
            for (sum = 0, i = 0; i < n; i++) sum += s[i].score;
            printf(n ? "avg=%.2f\n" : "空\n", n ? sum / n : 0);
        } else if (op == 7) {
            int j; Student t;
            for (i = 0; i < n - 1; i++)
                for (j = 0; j < n - 1 - i; j++)
                    if (s[j].score < s[j+1].score) { t = s[j]; s[j] = s[j+1]; s[j+1] = t; }
            printf("已按成绩降序\n");
        }
    }
    return 0;
}

```

项目3 参考实现（核心）

```

#include <stdio.h>
#include <string.h>

#define MAX 100
typedef struct {
    int id; char title[50]; char author[30]; int stock;
} Book;

```

```
int main(void) {
    Book b[MAX];
    int n = 0, op, i, id;
    char key[50];
    while (1) {
        printf("1加2查3借4还5列0退: ");
        scanf("%d", &op);
        if (op == 0) break;
        if (op == 1 && n < MAX) {
            scanf("%d %s %s %d", &b[n].id, b[n].title, b[n].author, &b[n].stock); n++;
        } else if (op == 2) {
            scanf("%s", key);
            for (i = 0; i < n; i++)
                if (strstr(b[i].title, key)) printf("%d %s 库存%d\n", b[i].id, b[i].title,
b[i].stock);
        } else if (op == 3) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (b[i].id == id) {
                if (b[i].stock > 0) { b[i].stock--; printf("借阅成功, 库存%d\n", b[i].stock); }
                else printf("无库存\n");
            }
        } else if (op == 4) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (b[i].id == id) { b[i].stock++; printf("归还成功\n"); }
        } else if (op == 5) {
            for (i = 0; i < n; i++)
                printf("%d %s %s %d\n", b[i].id, b[i].title, b[i].author, b[i].stock);
        }
    }
    return 0;
}
```

💡 学习建议

每日学习时间建议

时间段	活动	时长
上午	学习新知识、观看教学视频	2-3小时
下午	编写代码、完成作业练习	2-3小时
晚上	复习总结、阅读参考资料	1小时

学习方法

- 动手实践**：学习编程最好的方式是动手写代码，不要只看不动手
- 调试能力**：学会在 Dev-C++ 中设断点、单步执行，观察变量如何变化

3. **阅读代码**：阅读优秀的代码，学习他人的编程风格
4. **记录笔记**：记录重要知识点和遇到的问题
5. **交流讨论**：遇到问题不要独自埋头苦思，积极与同学交流

常见问题解决

- **编译错误**：仔细阅读 Dev-C++ 下方编译日志，根据行号检查语法和拼写
- **运行错误**：使用调试功能逐步执行，观察变量值的变化
- **逻辑错误**：画出流程图，分析程序的执行逻辑
- **环境问题**：回到 Day 1，检查编译器是否被正确检测到，项目/文件是否保存为 `.c`

学习目标检验

第一周检验

1. 能否用 Dev-C++ 独立编写、编译并运行一个C程序？
2. 能否使用变量、运算符和表达式进行计算，并理解常见易错点（如 `=` 与 `==`）？
3. 能否使用if-else和switch实现条件判断？
4. 能否使用while、do-while和for实现循环？
5. 能否完成每天的基础练习，并尝试至少一道挑战题？

第二周检验

1. 能否使用数组存储和处理批量数据？
2. 能否使用字符串处理函数？
3. 能否定义和调用函数？
4. 能否使用指针进行内存操作？
5. 能否使用结构体创建复杂数据结构？
6. 能否完成综合实战项目？

推荐学习资源

书籍

1. 《C程序设计语言》（K&R）- C语言经典教材
2. 《C Primer Plus》- 适合初学者的入门教材
3. 《C和指针》- 深入理解指针的权威著作

在线资源

1. [C语言中文网](#) - 中文教程
2. [LeetCode](#) - 编程练习平台
3. [CSDN](#) - 技术社区

视频教程

1. B站搜索"C语言教程"

加入学习交流群

扫码加入 **C 语言学习交流群**，一起答疑、打卡、交流作业思路：

群聊: C 语言学习交流群



该二维码 7 天内 (8月2日前) 有效，重新进入将更新

说明：微信群二维码通常有时效（图中标注 7 天内有效）。若过期无法加入，请重新获取最新二维码。

祝你学习愉快！编程之路从这里开始，加油！📌